NAQLA

المختبر 4

المختبر 4 — مساعد شحنة

هندسة الواجهات الخلفية بالوكلاء الذكيين

هندسة الواجهات الخلفية بالوكلاء الذكيين – بناء أنظمة خلفية قابلة للتوسع بالذكاء الاصطناعي ومن أجله

المختبر 4 – مساعد شحنة

نظرة عامة

الهدف

حتى اليوم، كان الذكاء الاصطناعي يعيش في الطرفية (Terminal). أما اليوم فينتقل *إلى داخل* النظام الخلفي (Backend). بنهاية المختبر سيتمكن عملاء «شحنة» من التحدث مع **«مساعد شحنة» (Shahna Assistant)** بالعربية أو الإنجليزية عبر `POST /api/agent/chat`: يطلبون سعراً، ويحجزون شحنة (بعد تأكيدهم فقط)، ويتتبعون شحنة أو يلغونها، ويطرحون أسئلة عن السياسات تُجاب من وثائق الشركة الحقيقية.

عادات اليوم:

1. **النموذج اللغوي الكبير (LLM) تبعية غير موثوقة.** غلّفه، وأعطه مخططاً (Schema)، وأعطه أدوات ضيقة النطاق، واختبره ببدائل مزيّفة (Fakes).
2. **الوكيل لا يعرف Prism.** سيكتب وكيل البرمجة شيفرة Prism من ذاكرته بثقة تامة. لكن Prism يتغيّر بين الإصدارات. الشيفرة المصدرية في `vendor/prism-php/prism` هي الحقيقة.

ما الذي ستبنيه

الجزء	مكانه
إعدادات النماذج	`config/shahna.php`
مصنّف النية (Intent Classifier) (Haiku، مخرجات مهيكلة)	`app/Agent/IntentClassifier.php`
مستخرج طلب عرض السعر من البريد الإلكتروني (مخرجات مهيكلة)	`app/Agent/ShipmentEmailExtractor.php`
وثائق السياسات مع البحث النصي الكامل	`app/Models/PolicyDocument.php`، والترحيل، والبذّارة
ست أدوات للوكيل	`app/Agent/Tools/`
الوكيل نفسه	`app/Agent/ShahnaAssistant.php`
ذاكرة المحادثة	النموذجان `AgentConversation` و`AgentMessage`
نقطة المحادثة (Endpoint)	`POST /api/agent/chat`

الجزء	مكانه
اختبارات تستخدم `Prism::fake()`	`tests/Feature/Agent/` (`IntentClassifierTest`، `ShipmentEmailExtractorTest`، `ToolsTest`، `AgentChatTest`)

نقطة البداية

```
git checkout day3-end
git switch -c my-day4
sail up -d
sail artisan migrate:fresh --seed
sail artisan test
```

يجب أن تنجح الاختبارات الـ 80 كلها قبل أن تبدأ. إن لم تنجح، فأخبر المدرّب الآن، لا في الساعة 15:30.

أضف مفتاح Anthropic API الخاص بك (يزوّدك به المدرّب) إلى `.env`:

```
ANTHROPIC_API_KEY=sk-ant-...
SHAHNA_AGENT_MODEL=claude-sonnet-5-5
SHAHNA_FAST_MODEL=claude-haiku-5-5
```

تحذير

لا تودِع `.env` أبداً، ولا تلصق مفتاحك في أي طلب للوكيل، ولا تضعه في أي اختبار. تحقّق من أن `.env` مدرج في `.gitignore` قبل أول إيداع (Commit) لك اليوم. كل اختبار في هذا المختبر يستخدم `Prism::fake()`؛ ولا يستهلك رموزاً (Tokens) حقيقية إلا ما تجريه يدوياً من استدعاءات Tinkerg curl.

توزيع الوقت

الجزء	الخطوة	الدقائق
الجزء الأول (11:00-12:30)	1. تحقّق من Prism وأضف الإعدادات، يدوياً	15
	2. أول استدعاء للنموذج اللغوي في Tinker، يدوياً	10
	3. IntentClassifier بمخرجات مهيكلة	25
	4. ShipmentEmailExtractor	20
	نقطة التحقق 1	20

الجزء	الخطوة	الدقائق
الجزء الثاني (15:45-14:15)	5. وثائق السياسات والبحث النصي الكامل	25
	6. الأدوات الست	25
	7. ShahnaAssistant وذاكرة المحادثة	20
	8. نقطة المحادثة والاختبارات اليدوية	15
	نقطة التحقق 2 والإيداع	5

الجزء الأول – التحدث مع نموذج لغوي من Laravel (11:00–12:30)

الخطوة 1 – تحقّق من Prism وأضف الإعدادات (15 دقيقة)

ماذا ولماذا. Prism (`prism-php/prism`) حزمة Laravel توفّر واجهة برمجية واحدة فوق مزوّدين كثيرين للنماذج اللغوية. نحن نستخدم مزوّد Anthropic فيها. وهي موجودة في `composer.json` منذ `day1-start` (الإصدار `^0.102`)، فلا حاجة إلى تثبيت شيء. ستتحقق منها وتكتب ملف الإعدادات بنفسك لتعرف بالضبط من أين تأتي أسماء النماذج.

> **تحذير**
>
> **يدوياً.** نفّذ هذه الخطوة دون الوكيل.

```
sail composer show prism-php/prism | head -3
```

افتح `vendor/prism-php/prism/config/prism.php` وابحث عن كتلة `anthropic` . تأكد أنها تقرأ `env('ANTHROPIC_API_KEY')` . لا تحتاج إلى نشر هذا الملف أو تغييره.

والآن أنشئ `config/shahna.php` :

```php
<?php

return [
    'agent_model' => env('SHAHNA_AGENT_MODEL', 'claude-sonnet-5-5'),
    'fast_model' => env('SHAHNA_FAST_MODEL', 'claude-haiku-5-5'),

    // Upper limit on model round-trips (tool calls + final answer) for one chat message.
    'max_steps' => 6,

    // How many earlier messages of a conversation are sent back to the model.
    'history_limit' => 20,
];
```

لماذا ملف إعدادات بدلاً من استدعاء `env()` داخل أصنافك؟ لأن `php artisan config:cache` في بيئة الإنتاج يجعل `env()` تعيد `null` خارج ملفات الإعدادات. الشيفرة تقرأ `config('shahna.agent_model')`؛ وملفات الإعدادات وحدها تقرأ `env()`.

```bash
sail artisan tinker --execute="dump(config('shahna'));"
```

```
array:4 [
  "agent_model" => "claude-sonnet-5-5"
  "fast_model" => "claude-haiku-5-5"
  "max_steps" => 6
  "history_limit" => 20
]
```

أضف سطراً واحداً إلى `CLAUDE.md` حتى يتوقف الوكيل عن التخمين:

```
- Prism API: read vendor/prism-php/prism/src before using any Prism class or method. Do not rely on memory. LLM tests must use Prism::fake().
```

الخطوة 2 — أول استدعاء للنموذج اللغوي، في Tinker (10 دقائق)

ماذا ولماذا. قبل أي تجريد (Abstraction)، شاهد الشيء الخام: طلب واحد، واستجابة واحدة، وعدد رموز، وتكلفة.

> **تحذير**
>
> **يدوياً.** اكتب هذا بنفسك في Tinker. بلا وكيل.

```bash
sail artisan tinker
```

```
> use Prism\Prism\Facades\Prism;
> use Prism\Prism\Enums\Provider;
> $r = Prism::text()
    ->using(Provider::Anthropic, config('shahna.fast_model'))
    ->withSystemPrompt('You are a Libyan logistics assistant. Answer in one sentence.')
    ->withPrompt('What is the largest port city in eastern Libya?')
    ->asText();
> $r->text;
= "Benghazi is the largest port city in eastern Libya."
> $r->usage->promptTokens;
= 31
> $r->usage->completionTokens;
= 14
> $r->finishReason;
= Prism\Prism\Enums\FinishReason {#... +name: "Stop"}
```

سيختلف نصّك وعدد الرموز لديك. شغّل الاستدعاء نفسه مرتين أخريين. هل الإجابة متطابقة؟ على الأرجح ليست متطابقة حرفياً. ولهذا لن نتحقق أبداً من نص حرّ في الاختبارات.

جرّب أيضاً أن تكتب طلب المستخدم نفسه بالعربية (مثلاً: «ما أكبر مدينة ساحلية في شرق ليبيا؟») وقارن اللغة التي يجيب بها النموذج.

افتح الآن `vendor/prism-php/prism/src/Text/Response.php` في محررك. ابحث عن الخصائص `usage` و`steps` و`toolCalls` و`messages`. ستحتاج إليها الأربع بعد الظهر، وأنت الآن تعرف أين توجد فعلاً.

الخطوة 3 — IntentClassifier بمخرجات مهيكلة (25 دقيقة)

ماذا ولماذا. معظم رسائل العملاء تحتاج إلى توجيه، لا إلى تشغيل كامل للوكيل: «هل هذا سؤال عن سعر، أم عن تتبع، أم شيء آخر؟» نموذج رخيص وسريع (Haiku) مع **المخرجات المهيكلة (Structured Output)** يعيد JSON مطابقاً لمخطط، فتحصل شيفرتك على مصفوفة لا على نثر تحتاج إلى تحليله.

العقد المستهدف:

```
{"intent": "quote", "language": "ar"}
```

قيمة `intent` واحدة من `quote|book|track|cancel|policy|other`؛ وقيمة `language` هي `ar|en`.

🤖 اسأل وكيلك الذكي

طلب الوكيل (Agent Prompt)

خطّط أولاً وانتظر موافقتي. فروق (Diffs) صغيرة.

أنشئ الصنف App\Agent\IntentClassifier بدالة classify(string $message): array{intent: string, language: string} يجب أن تستخدم Prism::structured() مع مزوّد Anthropic و config('shahna.fast_model'). ابنِ المخطط باستخدام ObjectSchema و EnumSchema من Prism: intent ضمن [quote, book, track, cancel, policy, other]، languageg ضمن [ar, en]، وكلاهما مطلوب.

قبل كتابة الشيفرة، افتح vendor/prism-php/prism/src/Schema/ObjectSchema.php وSchema/EnumSchema.phpg وTesting/StructuredResponseFake.php واقتبس توقيعات الباني (Constructor) التي ستستخدمها.

إذا أعاد النموذج قيمة intent أو language خارج التعداد، فارجع إلى ['intent' => 'other', 'language' => 'en'] – ولا ترمِ استثناءً أبداً.

اكتب اختبار Pest أولاً في tests/Feature/Agent/IntentClassifierTest.php باستخدام Prism::fake() مع StructuredResponseFake::make()->withStructured([...]). غطِّ: تصنيفاً عادياً (تحقّق من أن الطلب استخدم النموذج السريع)، وقيمة تعداد غير صالحة من النموذج (الرجوع إلى القيمة الافتراضية). شغّل الاختبارات وهي حمراء، ثم نفّذ، ثم اجعلها خضراء.

بالإنجليزية:

```
Plan first and wait for my approval. Small diffs.

Create App\Agent\IntentClassifier with a method
  classify(string $message): array{intent: string, language: string}
It must use Prism::structured() with the Anthropic provider and
config('shahna.fast_model'). Build the schema with Prism's ObjectSchema and
EnumSchema: intent in [quote, book, track, cancel, policy, other],
language in [ar, en], both required.

Before writing code, open vendor/prism-php/prism/src/Schema/ObjectSchema.php,
Schema/EnumSchema.php and Testing/StructuredResponseFake.php and quote the
constructor signatures you will use.

If the model returns an intent or language outside the enum, fall back to
['intent' ⇒ 'other', 'language' ⇒ 'en'] — never throw.

Write the Pest test first at tests/Feature/Agent/IntentClassifierTest.php
using Prism::fake() with StructuredResponseFake::make()→withStructured([...]).
Cover: a normal classification (assert the request used the fast model) and
an invalid enum value from the model (fallback). Run the tests red, then
implement, then green.
```

ما الذي تتحقق منه في مخرجات الوكيل.

- جيد: يقتبس الباني الحقيقي `new ObjectSchema(name:, description:, properties:, requiredFields:)` و `new EnumSchema(name:, description:, options:)` من ملفات vendor.
- جيد: تستدعي الاختبارات `Prism::fake([...])` قبل استدعاء المصنّف، وتُعاد البديلة المزيّفة من `Prism::fake()` حتى يتمكن الاختبار من التحقق من الطلب (مثلاً باستخدام `$fake->assertRequest(fn ($requests) => ...)`).
- جيد: الرجوع إلى القيمة الافتراضية يتحقق من خيارات التعداد في PHP، لا بالثقة بالنموذج.
- علامة تحذير: استيرادات غير موجودة، مثل `Prism\Prism\Schema\EnumSchema::make()` أو `Prism::structured()->withJsonSchema()`. ابحث عن الدالة بنفسك: `grep -rn "function withSchema" vendor/prism-php/prism/src`. إن لم يجد grep شيئاً، فالوكيل اختلقها.
- علامة تحذير: اختبار يتصل بالواجهة الحقيقية (بلا `Prism::fake()`)، أو اختبار يقرأ `ANTHROPIC_API_KEY`.

```
sail artisan test --filter=IntentClassifier
```

```
   PASS  Tests\Feature\Agent\IntentClassifierTest
  ✓ it classifies a message with the fast model
  ✓ it falls back to a safe default when the model returns something off-schema

  Tests:    2 passed
```

(هذه أسماء الاختبارات في الحل المرجعي. قد تختلف أسماؤك، وقد تكون لديك حالات أكثر.)

جرّبه الآن فعلياً (هذا يستهلك بضعة رموز):

```
sail artisan tinker --execute="dump(app(App\Agent\IntentClassifier::class)->classify('وين شحنتي
        SHN-7K2Q9X؟'));"
```

```
array:2 [
  "intent" => "track"
  "language" => "ar"
]
```

جرّب خمس رسائل من عندك، منها رسالة مخادعة ("I want to cancel my quote request and just ask about insurance")، وجرّب السؤال نفسه بالعربية أيضاً. دوّن أي تصنيف خاطئ؛ فسيصبح حالة تقييم (Eval Case) غداً.

الخطوة 4 — ShipmentEmailExtractor (20 دقيقة)

ماذا ولماذا. يتلقى فريق المبيعات في «شحنة» رسائل بريد مثل *"Salam, we need to move about 2 tons of tiles from Misrata to Sabha next week, flatbed preferred."* يحوّل المستخرج هذه الرسالة إلى طلب عرض سعر يستطيع `PricingService` استخدامه. الدرس المهم: **عندما لا تذكر الرسالة شيئاً، فالإجابة هي `null`، لا تخمين.**

العقد المستهدف:

```
{
  "origin_city_code": "MRA",
  "destination_city_code": "SEB",
  "vehicle_type": "flatbed",
  "weight_kg": 2000,
  "pickup_date": null,
  "notes": "Tiles"
}
```

🤖 اسأل وكيلك الذكي

طلب الوكيل

خطّط أولاً وانتظر الموافقة. الاختبارات أولاً. فرق صغير.

أنشئ App::extract(string $email): array التي تعيد origin_city_code وdestination_city_code وpickup_date (YYYY-MM-DD) وweight_kg وvehicle_type وnotes - استخدم Prism::structured() مع config('shahna.fast_model') - المخطط: ObjectSchema بخصائص StringSchema / NumberSchema، كلٌّ منها nullable: true وكلٌّ منها مدرج في requiredFields - طلب النظام (System Prompt): اذكر رموز المدن (من جدول cities) وقيم App؛ استخدم null لأي شيء لم يُذكر في الرسالة؛ ولا تخمّن أبداً - بعد أن يستجيب النموذج، تحقّق في PHP باستخدام Validator، تماماً كما تفعل مع مدخلات المستخدم: يجب أن تكون رموز المدن ونوع المركبة قيماً حقيقية (Rule::in)، وweight_kg بين 1 و25000، وpickup_date بصيغة date_format:Y-m-d. القيم غير الصالحة ترمي ValidationException. أعد weight_kg عدداً صحيحاً (int).

الاختبارات في tests/Feature/Agent/ShipmentEmailExtractorTest.php باستخدام StructuredResponseFake: رسالة كاملة، ورسالة بحقول ناقصة (المتوقع null لا تخمين)، واستجابة مزيّفة فيها رمز مدينة غير صالح "XYZ" (المتوقع ValidationException). وتحقّق أيضاً من أن طلب النظام المرسل إلى النموذج يحتوي كلمة "null".

بالإنجليزية:

```
Plan first and wait for approval. Tests first. Small diff.

Create App\Agent\ShipmentEmailExtractor::extract(string $email): array
returning origin_city_code, destination_city_code, vehicle_type, weight_kg,
pickup_date (YYYY-MM-DD) and notes.
- Use Prism::structured() with config('shahna.fast_model').
- Schema: ObjectSchema with StringSchema / NumberSchema properties, every
  one nullable: true and every one listed in requiredFields.
- System prompt: list the city codes (from the cities table) and the
  App\Enums\VehicleType values; use null for anything not stated in the
  email; never guess.
- After the model responds, validate in PHP with Validator, exactly like
  user input: city codes and vehicle type must be real values (Rule::in),
  weight_kg 1..25000, pickup_date date_format:Y-m-d. Invalid values throw
  a ValidationException. Return weight_kg as an int.

Tests in tests/Feature/Agent/ShipmentEmailExtractorTest.php using
StructuredResponseFake: a complete email, an email with missing fields
(expects null, not a guess), and a fake response containing an invalid
city code "XYZ" (expects a ValidationException).
Also assert the system prompt sent to the model contains the word "null".
```

ما الذي تتحقق منه.

- جيد: أنواع المركبات المسموح بها تأتي من `array_column(VehicleType::cases(), 'value')` ورموز المدن من جدول `cities`، لا من نسخة مكتوبة يدوياً في الشيفرة.
- جيد: تحقّق في جهة PHP بعد استدعاء النموذج. المخرجات المهيكلة تجعل JSON الصالح مرجّحاً؛ لكنها لا تضمن *قيماً صحيحة*.
- علامة تحذير: طلب النظام يقول "make a reasonable assumption if missing". هذا بالضبط السلوك الذي لا تريده حين يتعلق الأمر بسعر.
- علامة تحذير: المستخرج ينشئ `Quote` بنفسه. مهمته الاستخراج فقط؛ والتسعير يبقى في `PricingService`.

```
sail artisan test --filter=ShipmentEmailExtractor
```

```
  Tests:    3 passed
```

نقطة تحقق

نقطة التحقق 1 (12:30)

```
sail artisan test
```

```
  Tests:    N passed
```

`N` هو عدد اختباراتك في اليوم الثالث (80) مضافاً إليه 5 اختبارات جديدة على الأقل، مع **صفر إخفاقات** وبلا أي اتصالات شبكية (جرّب ذلك مع إطفاء Wi-Fi؛ يجب أن تنجح رغم ذلك).

تحقّق يدوي مع النموذج الحقيقي:

```
sail artisan tinker --execute="dump(app(App\Agent\ShipmentEmailExtractor::class)
       →extract('Hi, 800 kg of medicine from Tripoli to Benghazi, needs to stay
        cold.'));"
```

```
array:6 [
  "origin_city_code" ⇒ "TIP"
  "destination_city_code" ⇒ "BEN"
  "vehicle_type" ⇒ "reefer"
  "weight_kg" ⇒ 800
  "pickup_date" ⇒ null
  "notes" ⇒ "Medicine, must stay cold"
]
```

جرّب أيضاً رسالة بريد مماثلة بالعربية (مثلاً: «مرحباً، 800 كيلو أدوية من طرابلس إلى بنغازي، لازم تبقى باردة.») وقارن النتيجة.

ملاحظة: الانتقال من "needs to stay cold" إلى `reefer` استنتاج. قرّر مع زميلك هل هذا مقبول أم يجب أن تكون القيمة `null`. لا توجد إجابة صحيحة واحدة؛ لكن يجب أن توجد إجابة *مكتوبة* في طلب النظام لديك.

```
git add -A
git commit -m "feat: prism, intent classifier, email extractor"
```

الجزء الثاني – الأدوات والاسترجاع وحلقة الوكيل (14:15–15:45)

الخطوة 5 – وثائق السياسات والبحث النصي الكامل (25 دقيقة)

ماذا ولماذا. يجب أن يجيب المساعد عن أسئلة السياسات ("Can I ship gas cylinders?" أو «هل يمكنني شحن أسطوانات الغاز؟») من سياسات «شحنة» الحقيقية، لا من خيال النموذج. هذا هو **الاسترجاع (Retrieval)**: اعثر على النص ذي الصلة وضعه في الطلب. نستخدم البحث النصي الكامل (Full-Text Search) في PostgreSQL، فلا خدمة إضافية ولا مفتاح API إضافي.

أولاً، الجدول. اطلب من الوكيل الترحيل (Migration) والنموذج والبذّارة (Seeder):

🤖 اسأل وكيلك الذكي

طلب الوكيل

خطّط أولاً وانتظر الموافقة. فرق صغير.

1. ترحيل لجدول policy_documents: id، slugg (فريد)، titleg، bodyg (text)، والطوابع الزمنية. ثم، فقط عندما يكون 'pgsql' === ('DB::getDriverName() (الاختبارات تعمل على SQLite التي لا تدعم tsvector)، أضف باستخدام DB::statement search_vector tsvector GENERATED ALWAYS AS (setweight(to_tsvector('english', coalesce(title, '')), 'A') || setweight(to_tsvector('english', coalesce(body, '')), 'B')) STORED وفهرس GIN باسم policy_documents_search_vector_index على search_vector. يجب أن تحذف down() الجدول.
2. Appبالحقول القابلة للتعبئة slug titleg bodyg. يجب ألا يكون search_vector قابلاً للتعبئة (Postgres يحسبه)؛ أضفه إلى $hidden.
3. PolicyDocumentSeeder (updateOrCreate حسب slug) بخمس وثائق: cancellation، وprohibited-goods، insuranceg، delivery-timesg، paymentg. العناوين بالإنجليزية والعربية ("Cancellation Policy – سياسة الإلغاء")؛ والنصوص بالإنجليزية، بين 100 و200 كلمة لكل منها. يجب أن تطابق سياسة الإلغاء قواعد ShipmentStatus لدينا: مجانية في حالتي pending وconfirmed، وبرسوم بعد assigned، وغير ممكنة بعد picked_up. الأسعار والدفع بالدينار الليبي LYD (نقداً للسائق عند الاستلام أو بتحويل مصرفي).
4. سجّلها في DatabaseSeeder. لا تكتب شيفرة البحث بعد. شغّل migrate:fresh -seed واعرض المخرجات.

بالإنجليزية:

```
Plan first and wait for approval. Small diff.

1. Migration for policy_documents: id, slug (unique), title, body (text),
   timestamps. Then, only when DB::getDriverName() === 'pgsql' (tests run
   on SQLite, which has no tsvector), add with DB::statement
   search_vector tsvector GENERATED ALWAYS AS (
     setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
     setweight(to_tsvector('english', coalesce(body, '')), 'B')) STORED
   and a GIN index policy_documents_search_vector_index on search_vector.
   down() must drop the table.
2. App\Models\PolicyDocument with fillable slug, title, body.
   search_vector must NOT be fillable (Postgres computes it); add it to
   $hidden.
3. PolicyDocumentSeeder (updateOrCreate by slug) with 5 documents:
   cancellation, prohibited-goods, insurance, delivery-times, payment.
   Titles in English and Arabic ("Cancellation Policy — سياسة الإلغاء");
   bodies in English, 100-200 words each.
   Cancellation must match our ShipmentStatus rules: free in pending and
   confirmed, a fee once assigned, not possible after picked_up.
   Prices and payment are in LYD (cash to the driver at pickup or bank
   transfer).
4. Register it in DatabaseSeeder. Do not write the search code yet.
Run migrate:fresh --seed and show the output.
```

ما الذي تتحقق منه.

- جيد: إعداد البحث النصي `'english'` مع `setweight` (تطابقات العنوان تأتي في الترتيب قبل تطابقات النص). هذا الإعداد يردّ الكلمات الإنجليزية إلى جذورها (Stemming)، فتجد "cancelled" الكلمة "cancel". لكنه لا يفهم العربية، ولهذا تطلب أداة `search_policies` من النموذج بضع كلمات مفتاحية *إنجليزية*، حتى عندما يكتب العميل بالعربية.
- جيد: شيفرة SQL الخاصة بـ PostgreSQL وحده محمية بالشرط `DB::getDriverName() === 'pgsql'`، كما يشترط `CLAUDE.md`. من دون هذا الشرط، يفشل كل اختبار على SQLite.
- علامة تحذير: سياسة الإلغاء المزروعة في البذّارة تناقض التعداد لديك (مثلاً: "you can cancel while in transit"). يجب أن يتفق نص السياسة مع الشيفرة، وإلا فسيعِد الوكيل بأشياء ترفضها الواجهة البرمجية.
- علامة تحذير: ملء `search_vector` من PHP في مراقِب نموذج (Model Observer). العمود المولَّد يقوم بذلك أصلاً.

اكتب الآن استعلام البحث **يدوياً** في `psql` قبل أن توجد أي شيفرة PHP.

> **تحذير**
>
> **يدوياً.** اكتب شيفرة SQL هذه وشغّلها بنفسك. فهم الاستعلام هو الهدف.

```
sail psql
```

```
SELECT slug,
       ts_rank(search_vector, websearch_to_tsquery('english', 'cancel or fee')) AS rank
FROM policy_documents
WHERE search_vector @@ websearch_to_tsquery('english', 'cancel or fee')
ORDER BY rank DESC
LIMIT 3;
```

يجب أن تأتي وثيقة `cancellation` أولاً. تعتمد الصفوف وقيم الترتيب لديك على النص الذي زرعته. جرّب استعلاماً بلا `or` (`'cancel shipment fee'`: يجب أن تتطابق كل كلمة)، واستعلاماً مع `or` (`'insurance or damage'`)، واستعلاماً بالعربية (`'إلغاء'`: إعداد `english` لا يردّ الكلمات العربية إلى جذورها، فتوقّع نتائج ضعيفة أو لا نتائج). ثم تحقّق من أن الفهرس مستخدم:

```
EXPLAIN SELECT slug FROM policy_documents
WHERE search_vector @@ websearch_to_tsquery('english', 'insurance');
```

مع خمسة صفوف فقط قد يختار PostgreSQL المسح التسلسلي (Sequential Scan)؛ وهذا لا بأس به. أما مع آلاف الصفوف فسترى `Bitmap Index Scan on policy_documents_search_vector_index`.

لاحظ `websearch_to_tsquery`: فهي تقبل النص الحر من المستخدمين بأمان (علامات الاقتباس، و`or`، و`-word`) ولا ترمي خطأ صياغة أبداً، على عكس `to_tsquery`.

الخطوة 6 — الأدوات الست (25 دقيقة)

ماذا ولماذا. الأدوات (Tools) هي الطريقة الوحيدة التي يستطيع بها الوكيل أن *يفعل* شيئاً. كل أداة دالة ضيقة النطاق ومحدّدة الأنواع يستطيع النموذج طلبها. النموذج يختار *أي أداة* و*أي معاملات*؛ أما شيفرة PHP لديك فتقرّر *هل هذا مسموح*.

الأداة	الغرض	القاعدة
`get_quote`	تغلّف `PricingService` وتنشئ `Quote`	رموز المدن ونوع المركبة يُتحقق منها في PHP
`book_shipment`	تنشئ شحنة من عرض سعر	فقط إذا كان `confirmed: true`؛ ويجب أن يعود العرض للمستخدم وأن يكون غير منتهي الصلاحية وغير مستخدم
`track_shipment`	الحالة والأحداث حسب المرجع	شحنات المستخدم نفسه فقط
`cancel_shipment`	تلغي حسب المرجع	قواعد `canTransitionTo` نفسها التي تستخدمها الواجهة البرمجية
`search_policies`	بحث نصي كامل في `PolicyDocument`	أفضل 3 نتائج

الأداة	الغرض	القاعدة
`escalate_to_human`	تنشئ `SupportTicket`	تعيد معرّف تذكرة مثل `TKT-1`

🤖 اسأل وكيلك الذكي

طلب الوكيل

خطّط أولاً: اذكر صنف كل أداة ومعاملاتها وشكل ما تعيده، ثم انتظر الموافقة. ابنِ الأدوات واحدة أو اثنتين في كل مرة، مع الاختبارات، بفروق صغيرة.

أنشئ صنفاً لكل أداة في App: GetQuoteTool (get_quote)، BookShipmentTool (book_shipment)، TrackShipmentTool (track_shipment)، CancelShipmentTool (cancel_shipment)، SearchPoliciesTool (search_policies)، EscalateToHumanTool (escalate_to_human). كل صنف يرث Prism، ويأخذ المستخدم المصادَق عليه App في بانيه (SearchPoliciesTool لا يحتاج إلى مستخدم)، ويهيّئ نفسه هناك باستخدام $this->as(…)->for(…)->withStringParameter / withNumberParameter / withBooleanParameter / withEnumParameter(…)، وينفّذ عمله في __invoke(). اقرأ vendor/prism-php/prism/src/Tool.php أولاً واستخدم فقط الدوال الموجودة هناك.

المعاملات: - get_quote: origin_city_code، destination_city_code (تعدادان لرموز المدن)، vehicle_type (تعداد لقيم VehicleType)، weight_kg. - book_shipment: quote_id، pickup_at (ISO 8601)، confirmed (منطقي boolean). - track_shipment: reference. - cancel_shipment: reference، reason. - search_policies: query. - escalate_to_human: reason.

القواعد: - المستخدم يأتي دائماً من الباني. لا تأخذ أي أداة معاملاً باسم user_id. - أعد استخدام الشيفرة الموجودة: PricingService::createQuote لـ get_quote؛ BookingService::book لـ book_shipment؛ BookingService::cancel (الذي يستخدم قواعد canTransitionTo نفسها التي تستخدمها الواجهة البرمجية) لـ cancel_shipment. - book_shipment: إذا لم تكن confirmed صحيحة (true)، فأعد رسالة تطلب من النموذج أن يسأل العميل التأكيد، ولا تنشئ شيئاً. - track_shipment و cancel_shipment تبحثان عن الشحنة عبر $user->shipments() وتعيدان "No shipment with that reference was found for this customer." في غير ذلك (لا تكشف أنها موجودة). - search_policies تستخدم نطاق PolicyDocument::search()؛ على PostgreSQL websearch_to_tsquery('english', ?) مع ربط (Binding)، مرتبةً حسب ts_rank؛ وعلى SQLite (الاختبارات) بديل باستخدام LIKE. الحد 3. أعد العناوين والنصوص نصاً. - escalate_to_human تنشئ صفاً في SupportTicket (user_id، reasong، والحالة open) وتعيد JSON فيه ticket_id بالشكل "TKT-{id}". - تعيد الأدوات سلسلة JSON قصيرة عند النجاح، ورسالة نصية قصيرة عادية عندما ترفض أو تفشل، حتى يستطيع النموذج شرح ذلك.

الاختبارات في tests/Feature/Agent/ToolsTest.php: استدعِ كل أداة مباشرة باستخدام $tool->handle(…) (named args)، بالطريقة نفسها التي يستدعيها بها Prism (بلا نموذج لغوي). يجب أن تتضمن: book_shipment مع confirmed=false لا تنشئ أي شحنة؛ track_shipment لا ترى شحنة مستخدم آخر؛ cancel_shipment تتبع قواعد الواجهة البرمجية نفسها (ترفض شحنة في حالة in_transit)؛ search_policies تعيد وثيقة الإلغاء عند البحث عن "cancel".

بالإنجليزية:

```
Plan first: list each tool class, its parameters and its return shape, then
wait for approval. Build tools one or two at a time, with tests, small diffs.

Create one class per tool in App\Agent\Tools: GetQuoteTool (get_quote),
BookShipmentTool (book_shipment), TrackShipmentTool (track_shipment),
CancelShipmentTool (cancel_shipment), SearchPoliciesTool (search_policies),
EscalateToHumanTool (escalate_to_human). Each class extends
Prism\Prism\Tool, takes the authenticated App\Models\User in its
constructor (SearchPoliciesTool needs no user), configures itself there with
$this→as(...)→for(...)→withStringParameter / withNumberParameter /
withEnumParameter / withBooleanParameter(...), and does its work in
__invoke(). Read vendor/prism-php/prism/src/Tool.php first and use only
methods that exist there.

Parameters:
- get_quote: origin_city_code, destination_city_code (enums of the city
  codes), vehicle_type (enum of VehicleType values), weight_kg.
- book_shipment: quote_id, pickup_at (ISO 8601), confirmed (boolean).
- track_shipment and cancel_shipment: reference. search_policies: query.
  escalate_to_human: reason.

Rules:
- The user always comes from the constructor. No tool takes a user_id
  parameter.
- Reuse existing code: PricingService::createQuote for get_quote;
  BookingService::book for book_shipment; BookingService::cancel (which uses
  the same canTransitionTo rules as the API) for cancel_shipment.
- book_shipment: if `confirmed` is not true, return a message telling the
  model to ask the customer to confirm, and create nothing.
- track_shipment and cancel_shipment look the shipment up through
  $user→shipments() and return "No shipment with that reference was found
  for this customer." otherwise (do not reveal that it exists).
- search_policies uses a PolicyDocument::search() scope: on PostgreSQL
  websearch_to_tsquery('english', ?) with a binding, ordered by ts_rank; on
  SQLite (tests) a LIKE fallback. Limit 3. Return the titles and bodies as
  text.
- escalate_to_human creates a SupportTicket row (user_id, reason, status
  open) and returns JSON with ticket_id "TKT-{id}".
- Tools return a short JSON string on success and a short plain-text
  message when they refuse or fail, so the model can explain it.

Tests in tests/Feature/Agent/ToolsTest.php: call each tool directly with
$tool→handle(...named args), the same way Prism calls it (no LLM).
Must include: book_shipment with confirmed=false creates no shipment;
track_shipment cannot see another user's shipment; cancel_shipment follows
the same rules as the API (it refuses an in_transit shipment); search_policies
returns the cancellation document for "cancel".
```

ما الذي تتحقق منه.

- جيد: تستدعي الاختبارات الأداة مباشرة، مثلاً `(new GetQuoteTool($user))` `→handle(origin_city_code: 'TIP', ...)` وتتحقق من النتيجة ومن قاعدة البيانات. الأداة مجرد

صنف PHP؛ فاختبرها كأي صنف. (يجب أن تعيد `get_quote` للمسار MRA ← TIP بالمركبة `box_truck` ووزن 1000 كغ القيمة `"price_lyd": "322.00"`.)

- جيد: يستخدم `PolicyDocument::search()` الاستدعاء `whereRaw("search_vector @@ websearch_to_tsquery('english', ?)", [$tsQuery])` مع ربط. ابحث عن هذا بنفسك.
- علامة تحذير: `"... websearch_to_tsquery('english', '$query')"` مع دمج النص داخل السلسلة (String Interpolation). هذا حقن SQL (SQL Injection)، كتبه النموذج، وتطلقه رسالة من عميل.
- علامة تحذير: منطق الحجز منسوخ ولصوق من المتحكم (Controller) بدلاً من إعادة استخدامه. النسختان تتباعدان مع الوقت؛ وإحداهما ستنسى فحص «العرض مستخدم مسبقاً».
- علامة تحذير: أداة تقبل `user_id` أو `email` معاملاً. يمكن إقناع النموذج بتمرير معرّف أي شخص.

```
sail artisan test --filter=Tools
```

```
  Tests:    12 passed
```

(يحتوي `ToolsTest` المرجعي على 12 اختباراً. قد يختلف عددك؛ لكن كل قاعدة في الجدول أعلاه تحتاج إلى اختبار واحد على الأقل.)

الخطوة 7 — ShahnaAssistant وذاكرة المحادثة (20 دقيقة)

ماذا ولماذا. حلقة الوكيل (Agent Loop): أرسل المحادثة مع تعريفات الأدوات؛ فإذا طلب النموذج أداة، فشغّلها وأعد إليه النتيجة؛ وكرّر حتى يجيب النموذج نصاً، بحد أقصى **6 خطوات**. يشغّل Prism هذه الحلقة نيابةً عنك عندما تستدعي `withMaxSteps(config('shahna.max_steps'))`. مهمتك هي طلب النظام، وقائمة الأدوات، والذاكرة.

🤖 اسأل وكيلك الذكي

طلب الوكيل

خطّط أولاً وانتظر الموافقة. فروق صغيرة، والاختبارات أولاً.

1. الترحيلات والنموذجان AgentConversation (user_id، والطوابع الزمنية) و AgentMessage (conversation_id، roleg :user|assistant، contentg نصي، وtool_callsg من نوع json يقبل null، والطوابع الزمنية). حوّل tool_calls (Cast) إلى مصفوفة. أضف AgentMessage::toPrismMessage(). يمكن أن يكون جدول support_tickets من الخطوة 6 في الترحيل نفسه.
2. App::reply(User $user, string $message, ?AgentConversation $conversation = null): array تعيد [‘conversation_id’ => int, ‘reply’ => string, ‘actions’ => array].
   - حمّل الرسائل السابقة من المحادثة (آخر config(‘shahna.history_limit’)) وحوّلها إلى كائنات Prism من نوع UserMessage / AssistantMessage (تحقّق من البواني في vendor/prism-php/prism/src/ValueObjects/Messages).
   - Prism::text()->using(Provider::Anthropic, config(‘shahna.agent_model’))، ثم withSystemPrompt، withMessagesg (السجل + UserMessage الجديدة)، withToolsg (الأدوات الست المبنية لـ $user)، وwithMaxSteps(config(‘shahna.max_steps’))، ثم asText().
   - قواعد طلب النظام: أجب بلغة العميل (العربية أو الإنجليزية)؛ استخدم الأدوات للأسعار والحجوزات والتتبع والإلغاءات، ولا تختلق أرقاماً أبداً؛ اعرض دائماً عرض السعر واطلب تأكيداً صريحاً قبل استدعاء book_shipment مع confirmed=true؛ ولأسئلة السياسات استدعِ search_policies وأجب من نتائجها فقط؛ واستدعِ escalate_to_human عند عدم التأكد أو عندما يطلب العميل التحدث إلى شخص.
   - actions = عنصر {tool, arguments} واحد لكل استدعاء أداة، من جميع الخطوات ($response->steps، toolCallsg لكل Step؛ ToolCall->nameg و ToolCall->arguments()).
   - إذا كان النص النهائي فارغاً (بلوغ الحد الأقصى للخطوات)، فأجب برسالة احتياطية ثابتة.
   - احفظ رسالة المستخدم ورد المساعد (مع tool_calls).
3. ضع طلب النظام في دالة خاصة systemPrompt(User $user) أو في قالب Blade، حسب اختيارك، وأخبرني لماذا.

الاختبارات (tests/Feature/Agent/AgentChatTest.php، عبر نقطة النهاية من الخطوة 8) مع Prism::fake() TextResponseFakeg: تُنشأ محادثة جديدة وتُخزّن الرسالتان؛ واستدعاء ثانٍ بالمحادثة نفسها يرسل الرسائل السابقة، وطلب النظام، والحد الأقصى للخطوات، والأدوات الست إلى النموذج (تحقّق باستخدام $fake->assertRequest(…))؛ وبديل مزيّف من خطوتين مبني باستخدام ResponseBuilder TextStepFakeg (استدعاء أداة get_quote، ثم نص) يظهر في actions.

بالإنجليزية:

```
Plan first and wait for approval. Small diffs, tests first.

1. Migrations and models AgentConversation (user_id, timestamps) and
   AgentMessage (conversation_id, role: user|assistant, content text,
   tool_calls json nullable, timestamps). Cast tool_calls to array. Add
   AgentMessage::toPrismMessage(). The support_tickets table from Step 6
   can live in the same migration.
2. App\Agent\ShahnaAssistant::reply(User $user, string $message,
   ?AgentConversation $conversation = null): array
   returning ['conversation_id' => int, 'reply' => string, 'actions' => array].
   - Load the previous messages of the conversation (the last
     config('shahna.history_limit')) and map them to Prism UserMessage /
     AssistantMessage objects (check the constructors in
     vendor/prism-php/prism/src/ValueObjects/Messages).
   - Prism::text()->using(Provider::Anthropic, config('shahna.agent_model')),
     withSystemPrompt, withMessages (history + new UserMessage), withTools
     (the six tools built for $user), withMaxSteps(config('shahna.max_steps')),
     then asText().
   - System prompt rules: answer in the customer's language (Arabic or
     English); use tools for prices, bookings, tracking and cancellations,
     never invent numbers; ALWAYS show the quote and ask for explicit
     confirmation before calling book_shipment with confirmed=true; for
     policy questions call search_policies and answer only from its results;
     escalate_to_human when unsure or when the customer asks for a person.
   - actions = one {tool, arguments} item per tool call, from all steps
     ($response->steps, each Step's toolCalls; ToolCall->name and
     ToolCall->arguments()).
   - If the final text is empty (max steps reached), reply with a fixed
     fallback message.
   - Persist the user message and the assistant reply (with tool_calls).
3. Put the system prompt in a private systemPrompt(User $user) method or a
   Blade view, your choice, and tell me why.

Tests (tests/Feature/Agent/AgentChatTest.php, through the endpoint from
Step 8) with Prism::fake() and TextResponseFake: a new conversation is
created and both messages stored; a second call with the same conversation
sends the earlier messages, the system prompt, max steps and the six tools to
the model (assert with $fake->assertRequest(...)); a two-step fake built with
ResponseBuilder and TextStepFake (a get_quote tool call, then text) shows up
in actions.
```

ما الذي تتحقق منه.

- جيد: اختبار يتحقق من أن السجل يُرسل فعلاً. أخطاء الذاكرة صامتة: الوكيل «يعمل» لكنه ينسى كل شيء.
- جيد: `withMaxSteps(config('shahna.max_steps'))` موجودة. من دونها ينفّذ Prism خطوة واحدة فقط، ولا تصل نتائج الأدوات أبداً إلى إجابة نهائية.
- علامة تحذير: تُحمَّل المحادثة باستخدام `AgentConversation::find($id)` دون فحص للملكية. يستطيع أي مستخدم قراءة أي محادثة بتخمين المعرّفات. يجب أن تكون `$user` أو ما يعادلها `->agentConversations()->findOrFail($id)`.

- علامة تحذير: تُرسل المحادثة كاملة إلى الأبد. عشرون رسالة (`shahna.history_limit`) حدّ مقصود؛ اسأل الوكيل أين يُطبَّق.

نصيحة

`Prism::fake()` لا ينفّذ أدواتك: إنه ببساطة يعيد الاستجابات المزيّفة بالترتيب. البديل المزيّف متعدد الخطوات المبني باستخدام `ResponseBuilder` و `TextStepFake` *يصف* فقط استدعاءات الأدوات ونتائجها. لهذا تختبر الخطوة 6 الأدوات مباشرة، وتختبر الخطوة 7 التوصيلات (الطلب، والسجل، والحفظ، و `actions`). ومعاً يغطيان الوكيل دون استدعاء حقيقي واحد للواجهة.

الخطوة 8 — نقطة المحادثة والاختبارات اليدوية (15 دقيقة)

🤖 **اسأل وكيلك الذكي**

طلب الوكيل

خطّط أولاً. الاختبارات أولاً. فرق صغير.

أضف POST /api/agent/chat خلف auth:sanctum. - FormRequest باسم AgentChatRequest: message مطلوبة، نصية، بحد أقصى 2000؛ conversation_id عدد صحيح يقبل null. - متحكم قابل للاستدعاء (Invokable) باسم AgentChatController: حمّل المحادثة باستخدام $request->user()->agentConversations()->findOrFail(...) (محادثة شخص آخر تعيد 404)، واستدعِ ShahnaAssistant وأعد 200 مع {conversation_id, reply, actions}. اختبارات ميزات في tests/Feature/Agent/AgentChatTest.php مع Prism::fake(): 422 بلا message؛ 200 بشكل JSON الصحيح؛ و404 عندما يعود conversation_id لمستخدم آخر. ثم شغّل مجموعة الاختبارات كاملة.

بالإنجليزية:

```
Plan first. Tests first. Small diff.

Add POST /api/agent/chat behind auth:sanctum.
- FormRequest AgentChatRequest: message required string max 2000;
  conversation_id nullable integer.
- Invokable AgentChatController: load the conversation with
  $request->user()->agentConversations()->findOrFail(...) (someone else's
  conversation is a 404), call ShahnaAssistant and return 200 with
  {conversation_id, reply, actions}.
Feature tests in tests/Feature/Agent/AgentChatTest.php with Prism::fake():
422 without message; 200 with the right JSON shape; 404 when
conversation_id belongs to another user.
Then run the full test suite.
```

اختبرها الآن فعلياً. سجّل الدخول بصفتك عميلاً واحتفظ بالرمز المميّز (Token):

```
TOKEN=$(curl -s -X POST http://localhost/api/auth/login \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"email":"customer@shahna.test","password":"password"}' | jq -r .token)
```

(`customer@shahna.test` / `password` هو العميل التجريبي المزروع في البذّارة.)

سؤال عن السعر بالعربية:

```
curl -s -X POST http://localhost/api/agent/chat \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"message":"كم سعر شحن 800 كيلو من طرابلس إلى بنغازي بشاحنة صندوقية؟"}' | jq
```

```
{
  "conversation_id": 1,
  "reply": "سعر شحن 800 كيلوغرام من طرابلس إلى بنغازي بشاحنة صندوقية هو 1290.00 دينار ليبي (رقم العرض 31). العرض صالح لمدة 15 دقيقة. هل تريد تأكيد الحجز؟",
  "actions": [
    {"tool": "get_quote", "arguments": {"origin_city_code": "TIP", "destination_city_code": "BEN", "vehicle_type": "box_truck", "weight_kg": 800}}
  ]
}
```

سيختلف نص الرد ورقم العرض. تحقّق من السعر **يدوياً** باستخدام معادلة التسعير والمسافة المزروعة في البذّارة (من طرابلس إلى بنغازي 1020 كم): `50 + 1020 × 1.2 + 800 × 0.02 = 1290.00`. إذا لم يطابق الرقم في الرد نتيجة `get_quote`، فالنموذج يختلق الأسعار، وطلب النظام لديك يحتاج إلى عمل.

قاعدة التأكيد. تابع المحادثة نفسها، لكن اطلب الحجز دون تأكيد التفاصيل:

```
curl -s -X POST http://localhost/api/agent/chat \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"message":"Book it for tomorrow 9am","conversation_id":1}' | jq '.reply, .actions'
```

جيد: يعيد الرد ذكر السعر ووقت الاستلام ويسأل "Shall I confirm this booking?"؛ ولا تحتوي `actions` على `book_shipment` مع `confirmed: true`. ثم أرسل `{"message":"Yes, confirm","conversation_id":1}` وتحقّق من أن شحنة قد أُنشئت الآن:

```
sail artisan tinker --execute="dump(App\Models\Shipment::latest()->first()?
      ->only('reference','status','price_lyd'));"
```

جرّب السؤال نفسه بالعربية أيضاً في محادثة جديدة (مثلاً: «احجزها لبكرة الساعة 9 الصبح»، ثم «نعم، أكّد»)، وتحقّق من أن قاعدة التأكيد تصمد باللغتين.

سؤال عن السياسات بالإنجليزية:

```
curl -s -X POST http://localhost/api/agent/chat \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"message":"Can I cancel after the truck picked up my goods?"}' | jq '.reply, .actions'
```

المتوقع: تتضمن `actions` الأداة `search_policies`، ويقول الرد إن الإلغاء غير ممكن بعد الاستلام. قارنه كلمة بكلمة مع السياسة التي زرعتها.

جرّب السؤال نفسه بالعربية أيضاً («هل يمكنني الإلغاء بعد أن تستلم الشاحنة بضاعتي؟»)، وقارن الكلمات المفتاحية التي يمرّرها الوكيل إلى أداة البحث في الحالتين.

أخيراً، تأكد من أن سلوك التأكيد محميّ باختبار. ولأن `Prism::fake()` لا يشغّل الأدوات، فمكان هذا الاختبار هو مستوى الأداة: يجب أن يستدعي `ToolsTest` الأداة `book_shipment` مع `confirmed: false` ويتحقق من أنه لم تُنشأ أي شحنة.

نقطة تحقق

نقطة التحقق 2 (15:45)

```
sail artisan test
```

```
   PASS  Tests\Feature\Agent\AgentChatTest
   PASS  Tests\Feature\Agent\IntentClassifierTest
   PASS  Tests\Feature\Agent\ShipmentEmailExtractorTest
   PASS  Tests\Feature\Agent\ToolsTest
   ...
  Tests:    N passed
```

صفر إخفاقات، وصفر استدعاءات حقيقية للواجهة في مجموعة الاختبارات. يحتوي المرجع `day4-end` على 104 اختبارات (80 من اليوم الثالث و24 اختباراً جديداً).

فحوص سريعة للواجهة البرمجية:

```
curl -s -X POST http://localhost/api/agent/chat -H "Accept: application/json" \
  -H "Content-Type: application/json" -d '{"message":"hi"}'
```

```
{"message":"Unauthenticated."}
```

```
curl -s -X POST http://localhost/api/agent/chat -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json" -H "Content-Type: application/json" -d '{}'
```

```
{"message":"The message field is required.","errors":{"message":["The message field is
        required."]}}
```

```
git add -A
git commit -m "feat: shahna assistant with tools, retrieval and memory"
```

إذا علقت

الحل المرجعي هو الوسم (Tag) `day4-end`:

```
git diff --stat day4-end
git show day4-end:app/Agent/ShahnaAssistant.php
git diff day4-end -- app/Agent/Tools/
git show day4-end:database/seeders/PolicyDocumentSeeder.php
```

أكثر العوائق شيوعاً وأين تبحث:

العَرَض	السبب المرجّح
`Class "Prism\Prism\..." not found`	الوكيل اختلق صنفاً. نفّذ `ls vendor/prism-` `php/prism/src` وابحث عنه.
الوكيل يرد لكنه لا يستخدم الأدوات أبداً	`withTools()` مفقودة، أو `withMaxSteps()` بقيت على 1
الاختبارات بطيئة أو تحتاج إلى الشبكة	أحد الاختبارات ينقصه `Prism::fake()`
البحث بالعربية لا يعيد شيئاً	متوقع: إعداد `'english'` لا يفهم العربية. الأداة تطلب من النموذج كلمات مفتاحية إنجليزية
كل اختبار يفشل بخطأ SQL متعلق بـ `tsvector`	شيفرة SQL الخاصة بـ PostgreSQL وحده في الترحيل غير محمية بالشرط `DB::getDriverName() === 'pgsql'`
`401` من Anthropic في Tinker	`ANTHROPIC_API_KEY` مفقود؛ شغّل `sail artisan` `config:clear` بعد تعديل `.env`

لإعادة ضبط فرعك على المرجع (يبقى عملك في `my-day4`):

```
git stash
git switch -c my-day4-reset day4-end
sail composer install
sail artisan migrate:fresh --seed
sail artisan test
```

أهداف إضافية (Stretch Goals)

1. **التوجيه حسب النية.** استدعِ `IntentClassifier` أولاً في متحكم المحادثة. للنية `policy`، استدعِ `search_policies` مباشرة وأجب باستخدام Haiku دون أي أدوات أخرى. قِس عدد استدعاءات Sonnet التي وفّرتها على مدى 10 رسائل. (يصبح هذا إلزامياً غداً.)
2. **نقطة لاستقبال البريد الإلكتروني.** أضف `POST /api/quotes/from-email` التي تشغّل `ShipmentEmailExtractor`، وتعيد `422` مع قائمة الحقول الناقصة عندما تكون أي قيمة `null`، وإلا فتنشئ عرض سعر عبر `PricingService`.
3. **pgvector.** أضف الامتداد `pgvector`، وخزّن التضمينات (Embeddings) لوثائق السياسات، وقارن أفضل 3 نتائج مع البحث النصي الكامل لعشرة أسئلة. ما الأسئلة التي تخطئ فيها كل طريقة؟
4. **البحث بالعربية.** افحص `SELECT cfgname FROM pg_ts_config;` بحثاً عن إعداد `arabic`. أضف نصاً عربياً إلى إحدى السياسات، وجرّب الإعدادين `'simple'` و`arabic`، وقارن النتائج لـ `'إلغاء'` مقابل `'الإلغاء'`.

التأمل

1. ما صنف Prism أو دالته التي أخطأ فيها وكيل البرمجة اليوم، وكيف اكتشفت ذلك (خطأ، أو `grep` في `vendor/`، أو بحث التوثيق في Boost)؟ ماذا كان سيحدث لو أن الشيفرة الخاطئة صادف أنها عملت؟
2. اذكر كل موضع تفرض فيه *شيفرة PHP لديك*، لا النموذج، قاعدةً ما (الملكية، والتأكيد، وقيم التعداد، وروابط SQL). أيٌّ من هذه سينهار لو وثقت بمعاملات النموذج؟
3. كل اختباراتك تستخدم بدائل مزيّفة. ما الذي *لا تستطيع* أن تخبرك به عن المساعد الحقيقي، وكيف ستعرفه؟ (تلميح: هذا هو `agent:eval` غداً.)